Algorithm for file updates in Python

Project description

At a health care organisation, part of my job was to create an algorithm that uses Python code to check whether the allow list contains any IP addresses identified on the remove list. If so, I should remove those IP addresses from the file containing the allow list. I created an algorithm to automate updating the `"allow_list.txt"` file and remove these IP addresses that should no longer have access.

Open the file that contains the allow list

Firstly, I assigned the file name as a string to the `import_file` variable:

```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"
```

Then I opened the file using the with statement:

```
# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:
```

In the algorithm, the `with` statement is used to help manage the resources used by closing the file after exiting the with statement. It is used alongside the `open()` function, this function has two parameters. The first parameter which is `import_file`, specifies the file path to the file, and the `"r"` indicates that I want to read the file. The code also uses the `as file` keyword to store the output of the `open()` function in a variable named `file`, while i work within the `with` statement.

Read the file contents

To read the opened file contents, I used the `.read()` method to convert it into a string. Then I assigned the string output from the method to the variable, `ip_addresses`.

```
with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

  ip_addresses = file.read()
```

In summary, this code reads the contents of the `"allow_list.txt"` file into a string format that allows me to later use the string to organize and extract data in my Python program.

Convert the string into a list

In order to work with the string, I converted the string into a list using the `.split()` function, this made it easier to access each IP address in the list.

```
# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()
```

By default, the `.split()` function splits the text by whitespace into list elements. In this algorithm, the `.split()` function takes the data stored in the variable `ip_addresses`, which is a string of IP addresses that are each separated by a whitespace, and it converts this string into a list of IP addresses. To store this list, I reassigned it back to the variable `ip_addresses`.

Iterate through the remove list

The main part of my algorithm involves iterating through the `remove_list`, using the `for` loop:

```
# Build iterative statement
# Name loop variable `element`
# Loop through `remove_list`

for element in remove_list:
```

Using the `for` loop in Python algorithm is to apply specific code statements to all elements in a sequence. The `for` keyword starts the `for` loop. It is followed by the loop variable `element` and the keyword `in`. The keyword `in` indicates to iterate through the sequence of IP addresses in the `remove_list` and assign each value to the loop variable `element`.

Remove IP addresses that are on the remove list

The algorithm checks each element to see if it is in the `ip_addresses` list, and removes it.

```
for element in remove_list:

  # Create conditional statement to evaluate if `element` is in `ip_addresses`

    if element in ip_addresses:

      # use the `.remove()` method to remove
      # elements from `ip_addresses`

        ip_addresses.remove(element)
```

First, within my `for` loop, I created an `if` statement that evaluated whether or not the loop variable `element` was found in the `ip_addresses` list. I did this because applying `.remove()` to elements that were not found in ip_addresses would result in an error.

Then, within that `if` statement, I applied `.remove()` to `ip_addresses`. I passed in the loop variable `element` as the argument so that each IP address that was in the `remove_list` would be removed from `ip_addresses`.

Update the file with the revised list of IP addresses

With the new revised list of IP addresses, I needed to update the allow list file.

```
# Convert `ip_addresses` back to a string so that it can be written into the text file

ip_addresses = "\n".join(ip_addresses)
```

Firstly, I used the `.join()` method to create a string from the list `ip_addresses` so that I could pass it in as an argument to the `.write()` method when writing to the file `"allow_list.txt"`. I used the string `("\n")` as the separator to instruct Python to place each element on a new line.

Then, I used another `with` statement and the `.write()` method to update the file:

```
# Build `with` statement to rewrite the original file

with open(import_file, "w") as file:

  # Rewrite the file, replacing its contents with `ip_addresses`

  file.write(ip_addresses)
```

In the `with` statement, instead of using the `"r"` as argument. I used a second argument of `"w"` with the `open()` function in my `with` statement. This argument indicates that I want to open a file to write over its contents. When using this argument `"w"`, I can call the `.write()` function in the body of the `with` statement. The `.write()` function writes string data to a specified file and replaces any existing file content.

Summary

I wrote an algorithm with Python, to update an approved IP address list (`"allow_list.txt"`) by filtering out addresses listed in a `remove_list`. First, I opened the text file, read its contents as a string, and parsed it into a list called `ip_addresses`. I then iterated through each address in `remove_list`, checking if it existed in `ip_addresses` list, and used `.remove()` to strip it out when matched. Finally, I used `.join()` to format the cleaned list back into a single string and overwrote the original `allow_list.txt` file.